

DATA 266 Homework 4

DATA 266 — Homework 4

Sneha Singh

SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

Notebook: `mini_gpt.ipynb`. Dataset: `course shakespeare.txt` (balcony scene from week-4 demo files).

I built a small decoder-only GPT with character tokens. No `nn.Transformer`, no `nn.MultiheadAttention`, no HuggingFace. I only used `nn.Linear`, `nn.LayerNorm`, `nn.GELU`, and embeddings.

Part 1 — Data

Char vocab size **49**. File length **1738** characters. Sequence length **128**.

- Input: tokens `[t ... t+127]`
- Target: tokens `[t+1 ... t+128]` (next character)

I showed two decoded windows in the notebook (starts 0 and 500). Target is always input shifted by one character.

Part 2 — Model

Manual multi-head causal attention (**4 heads**). Future positions get `-inf` before softmax. Each decoder block: `LayerNorm` → attention → residual, then `LayerNorm` → `Linear` → `GELU` → `Linear` → residual. Stack of **2** blocks. Token embedding + learnable positional embedding + final projection to vocab.

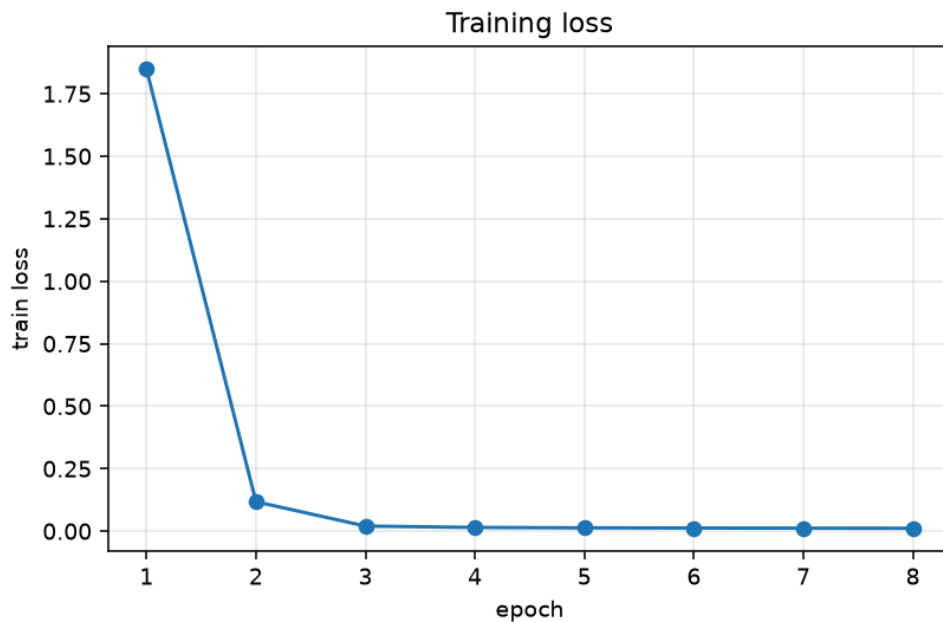
Field	Value
Hidden dim	128
Heads	4
Layers	2
Parameters	425,728

Part 3 — Training

Adam, learning rate $3e-4$, batch size 64, **8** epochs (assignment range 5–8; 300 random windows each epoch on CPU).

Epoch	Train loss
1	1.849
2	0.118
3	0.020
4	0.015
5	0.013
6	0.012
7	0.012
8	0.012

Loss went from about **1.85 down to 0.012**. The course file is short, so the model mostly memorizes it after a couple of epochs.



Training loss vs epoch

Part 4 — Sampling (prompt ROME0:)

I generated 200 new characters for each method.

Greedy — starts by repeating the balcony lines from the training text, then drifts into broken characters. Deterministic and fairly “stuck” on the memorized opening.

Temperature 0.2 — almost the same as greedy (still peaked).

Temperature 0.7 — same memorized opening, then messier spelling / spacing.

Temperature 1.2 — same opening, then much noisier letters and punctuation (clearest difference vs 0.2 / 0.7).

Top-k = 5 — narrower choices after the memorized start.

Top-k = 50 — wider mix / more odd fragments than k=5.

Part 5 — Compare the three techniques

Method	What it does	On my run
Greedy	always take argmax	Most stable; repeats training opening

Method	What it does	On my run
Temperature	divide logits by T before softmax	Low T $\approx$ greedy; high T = wilder tail
Top-k	sample only among k best chars	Small k safer; large k more variety

Most coherent: greedy decoding. It always picks the highest-probability next character, so the text is more stable and predictable. Temperature **0.2** looked almost the same (very peaked), but greedy is the clearest single answer.

Most diverse: temperature sampling with T = 1.2. The flatter distribution lets rarer characters through. Top-k **50** is also mixed, but **1.2** was the wildest on my run.

With this short course file, all methods still latch onto the memorized ROMEO: opening first; differences show more in the tail.